

Computer Science & IT

Database Management System



**File organization
And
Indexing**

Lecture No. 05



By- Vishal Sir

Recap of Previous Lecture



Topic

Dynamic Multi level index



Topic

Structure of B tree



Topic

Structure of B+ tree



Topics to be Covered



Topic

Dynamic Multi level index

Topic

Structure of B tree

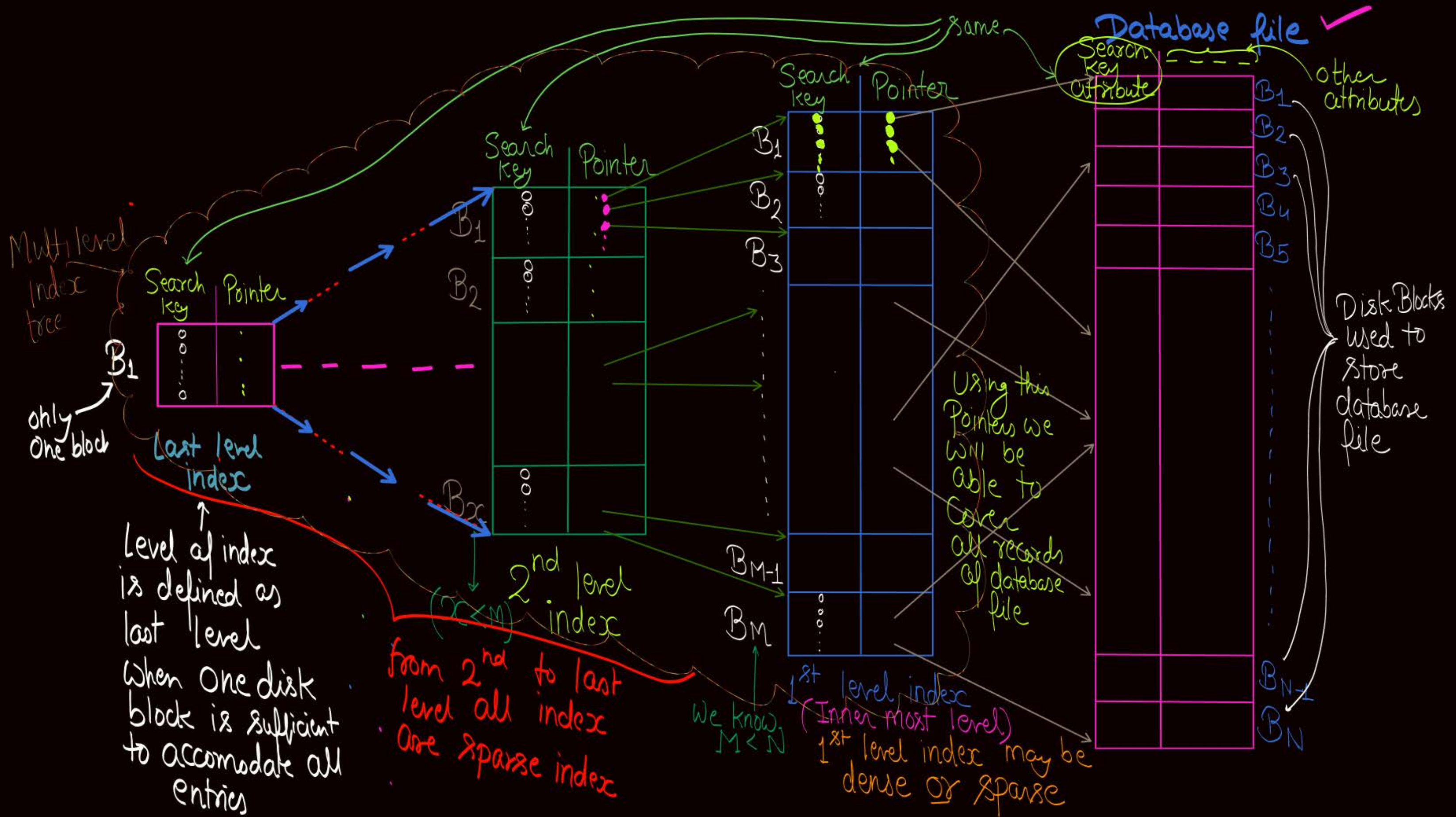
Topic

Structure of B+ tree



Topic : Multi-level Index

- Multilevel index is used to reduce the IO Cost w.r.t single level index



Note:- Multi level index can reduce the IO cost significantly.

$$\text{IO Cost using multi-level Index} = \underbrace{\text{No. of levels in multi level index tree}} + \underbrace{1}$$

from each level of index tree, exactly one block will be access from secondary memory to main memory

1 Block of database file will be access from secondary memory to main memory in order to access the record corresponding to the search key



Topic : Multi-level Index

Using multi level index

$$\text{IO Cost} = \text{No. of levels} + 1$$

↳ for all access

{ Best case, worst case, avg case }
all same



Topic : Multi-level Index

In multi-level index,

- ① 1st level of index may be dense or may be sparse
- ② Except first level, all other levels of index are always sparse index

Note:- "Multi-level index" is suitable only for static database {i.e. the database in which insertion and/or deletion are rare}

But if database happens to be dynamic {i.e. insertion/deletion are frequent} then after every insertion or deletion we may have to restructure all the levels of multi-level index.
∴ Most of the time will be wasted in maintaining the multi-level index itself.

Note:- For dynamic database we use
dynamic multi-level search trees
i.e. B tree
B⁺ tree



Topic : Dynamic Multi-level Search Tree

→ B tree
→ B+ tree } These are
Balanced multi-way
Search tree

ie. all leaf nodes
are at same
level

it may be
m-ary search
tree where
'm' can be
any value



Topic : B Tree



Order of a node of B tree :- In general, Order of a node of B tree is defined as Maximum number of Child pointer a node of B tree can have

Consider a B tree of order = P { i.e.,

Structure of an internal node of B tree

- Maximum number of Child pointer a node can have = P
- Maximum number of Key values a node can have = $(P-1)$

All Keys within a node will always be in ascending order

Search Key value $\rightarrow$ Record pointer corresponding to search key value K_1 $\rightarrow$ $\{ \therefore K_1 < K_2 < K_3 \dots < K_{P-1} \}$



Points to the Node of B tree at lower level that contains the information about Key value $< K_1$

$K_1 < \text{Key} < K_2$
Value

$K_2 < \text{Key} < K_3$
Value

$K_{P-2} < \text{Key} < K_{P-1}$
Value

$K_{P-1} < \text{Key}$
Values

Record Pointer :- Record pointers are used to point to the record in the database file corresponding to the associated key value.

Block Pointer :- Block pointers are used to point to a block(node) of B tree at lower level.
Also known as Child pointer / Node pointer / Tree pointer

Consider a B tree of order = P { i.e.,

Structure of a leaf node of B tree

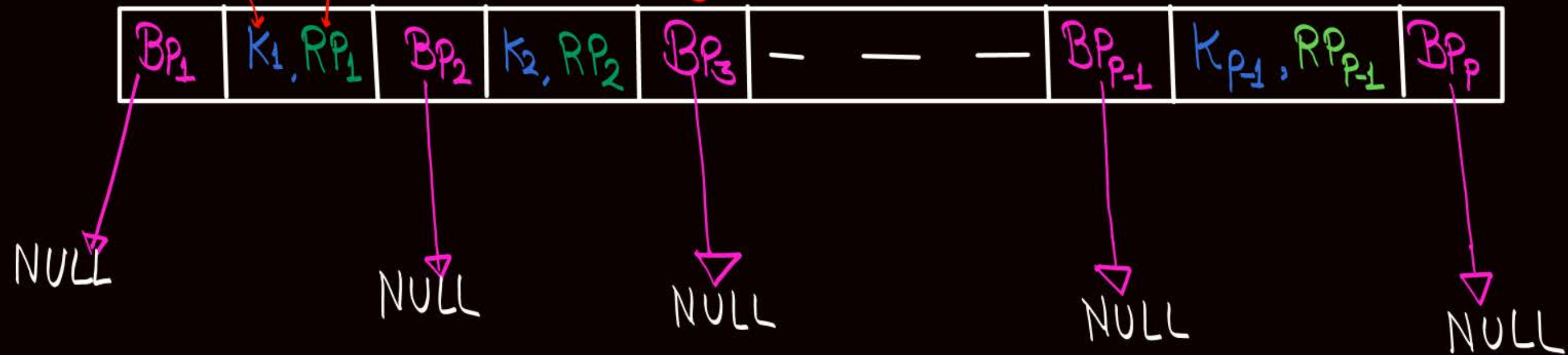
- Maximum number of Child pointer a node can have = P
- Maximum number of Key values a node can have = $(P-1)$

All keys within a node will always be in ascending order

Search Key value

Record pointer corresponding to search key value K_1

$$\{ \therefore K_1 < K_2 < K_3 \dots < K_{P-1} \}$$



Note:-

In B tree record pointers corresponding to the key values are present in leaf node of B tree, as well as in internal node of B tree

i.e., Structure of B tree is different from structure of normal multi-level index tree.

(In regular multi-level index tree all the record pointers are only at leaf level)

Note:-

In B tree each key value of database file will be present in exactly one node of B tree along with the record pointer corresponding to that key value, whereas in regular multi-level index tree a key value may be present in more than one index block

Note :-

Each node of B tree is a "disk block"
∴ Whatever information we want to store in a single node of B tree must fit within a single block of disk.

Consider a B tree of order = P } i.e.,

- Maximum number of Child pointer a node can have = P
- Maximum number of Key values a node can have = $(P-1)$

It is one node
of a B tree
of order = P



∴ it must fit
within the
block of disk

$$\Rightarrow \left\{ (P * \text{Block Pointer Size}) + (P-1) * (\text{Search Key Size} + \text{Record Pointer Size}) \right\} \leq \text{Disk Block Size}$$

↳ Using this equation we can obtain the maximum possible value of " P " (order)



Topic : B Tree

ie. 'K' is a search key

Consider a relations R with a key filed K. A B-tree of order P is used to access structures on K. Where 'P' denotes the maximum Number of tree pointers in a B-tree index node. Assume that key (k) is 30 bytes long disk block size 1024 byte. Each data pointer size P_D is 12 bytes long & each block pointer P_B is 10 byte long. In order for each B-Tree Node to fit in a single disk block, the maximum value of P is-

A 19

C X 21

B 20

D X 22

$$(P \times \text{Block Pointer} + (P-1) \times (\text{key size} + \text{Record Pointer size})) \leq \text{Block size}$$

$$P \times 10 + (P-1) \times (30 + 12) \leq 1024$$

$$52P \leq 1024 + 42$$

$$P \leq \frac{1066}{52} \Rightarrow P \leq 20.5 \dots$$

Hence $P \leq 20$ $\therefore$ Max Value = 20

Important

* Consider a B tree of order = 'P'

- ① Maximum number of Child Pointer a node of B tree can have is = P
(Block Pointer)
- ② Maximum number of Key values/Record pointers a node of B tree can have is = (P-1)
- ③ Minimum number of Child Pointer a root node of B tree must have is = 2
(Block Pointer)
- ④ Minimum number of Key values/Record pointers root node of B tree must have is = 1
- ⑤ Minimum number of Child Pointer a non-root node of B tree must have is = $\lceil \frac{P}{2} \rceil$
(Block Pointer)
- ⑥ Minimum number of Key values/Record pointers non-root node of B tree must have is = $(\lceil \frac{P}{2} \rceil - 1)$

it states that
index block must be
at least half filled {except root node}

HoWo

#Q.



a) Find the minimum number of nodes required in B tree of order = p and height = h

b) Find the minimum number of child pointers required in B tree of order = p and height = h

c) Find the minimum number of keys required in B tree of order = p and height = h

d) Find the maximum number of nodes possible in B tree of order = p and height = h

e) Find the maximum number of child pointers possible in B tree of order = p and height = h

f) Find the maximum number of keys possible in B tree of order = p and height = h

Consider
root node
is at
height = 0



Topic : B+ tree visualization

Structure of B+ tree is close to the structure of regular multi-level index tree.

Then Record with key value 3 will be in left sub-tree

Record with the value in an internal node is in right sub-tree

No record pointer is stored in internal node

Keys in internal node are dummy keys they are used only for searching the appropriate leaf node

if we say $key \leq 3$

$key < 3$

$3 \leq key < 5$

$5 \leq key$

leaf level

1	RP ₁	2	RP ₂	BP
---	-----------------	---	-----------------	----

3	RP ₃	4	RP ₄	BP
---	-----------------	---	-----------------	----

5	RP ₅	6	RP ₆	BP
---	-----------------	---	-----------------	----

NULL

Used to point the next block at leaf level

Search key field

other attributes

2
1
3
5
6
4

Database file

Record Pointer Corresponding to key value = 1

All the keys of database file are present at leaf level along with their record pointers



Topic : B+ Tree



- ✓ • B+ tree is somewhat similar to normal multi-level index.
- ✓ • In B+ Tree all keys along with their record pointers are present at leaf nodes of B+ Tree.
- ✓ *Keys within the node of B+ tree are store in ascending order*
- ✓ • In B+ Tree keys in the internal nodes are used only for search operation (they are dummy keys), they are not used to point any record of the database as no record pointer is available in internal nodes of B+ Tree.

Structure of an internal node of B+ tree

- All keys within a node will always be in ascending order

$$\{ \therefore K_1 < K_2 < K_3 \dots < K_{q-1} \}$$

Search Key value



Points to the Node of B+ tree at lower level that contains the information about Key value $< K_1$

$$K_1 \leq \text{Key} < K_2$$

Value

$$K_2 \leq \text{Key} < K_3$$

Value

$$K_{p-2} \leq \text{Key} < K_{p-1}$$

Value

$$K_{p-1} \leq \text{Key} < K_p$$

Value



Topic : Order of Internal node of B+ Tree

- Order of an internal node of B+ tree is defined as the maximum number of child pointer (block pointer/ node pointer) a B+ tree node can have. (same as B Tree)

an internal

- Let order of ^{an internal} node of B+ tree is denoted by 'q'

* Every information that we want to store in an internal node of B+ tree must fit in a single disk block

∴ If order of an internal node of B+ tree is = q

then,

$$\left\{ (q * \text{Block Pointer Size}) + ((q-1) * \text{Key Size}) \right\} \leq \text{Disk block size}$$



Topic : Order of Internal node of B+ Tree

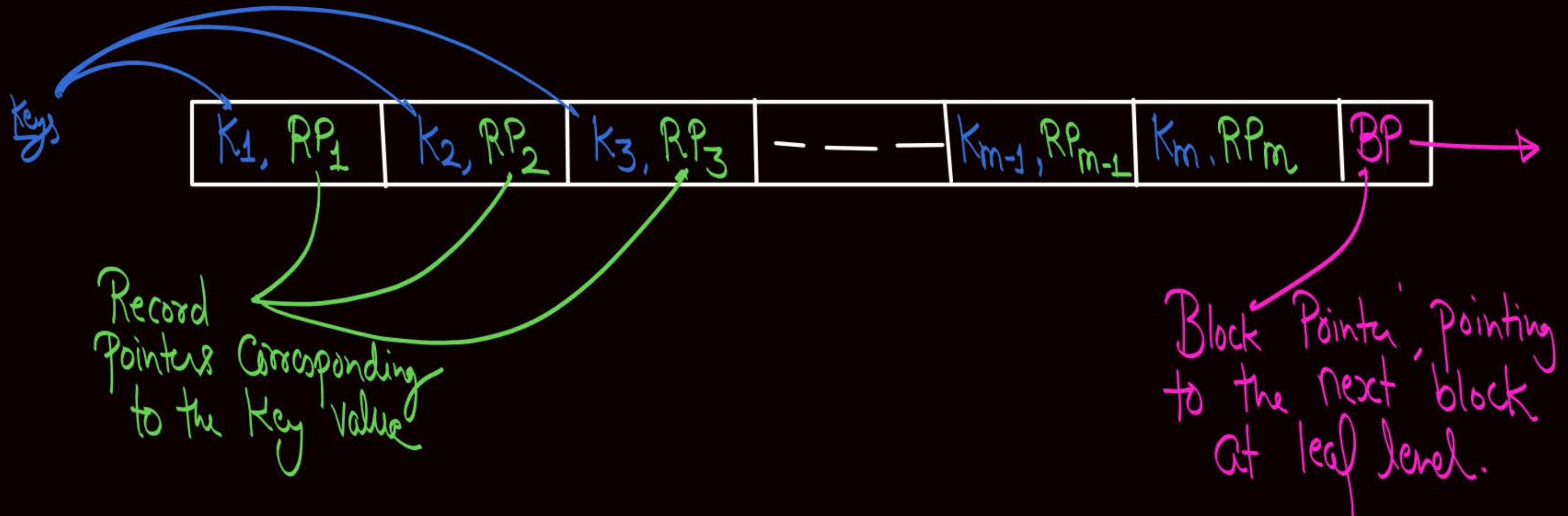
Let 'q' is the order of an internal node of B+ Tree

- Maximum number of child pointer ^{On internal} node can have is $= q$
- Maximum number of keys ^{On internal} node can have is $= q-1$
- Minimum number of child pointer a non-root ^{internal} node ^{must} have is $= \lceil q/2 \rceil$
- Minimum number of keys a non-root ^{internal} node ^{must} have is $= \lceil q/2 \rceil - 1$
- Minimum number of child pointer root node can have is $= 2$
- Minimum number of keys root node can have is $= 1$

When root node is an internal node

Structure of leaf node of B+ tree

- All keys within a node will always be in ascending order.





Topic : Order of leaf node of B+ Tree

- Order of leaf node of B+ tree is defined as the maximum number of keys(record pointers) a B+ tree node can have. (different from B Tree)
- * Let order of a node of B+ tree is denoted by 'm' { i.e. the maximum number of keys a leaf node can have is 'm' }
- * Everything that we want to store in a leaf node of B+ tree must fit into a single disk block
- * If order of leaf node of B+ tree is given as 'm', then

$$\left\{ m \times \left(\begin{array}{c} \text{Search key} \\ \text{size} \end{array} + \begin{array}{c} \text{Record Pointer} \\ \text{size} \end{array} \right) + 1 \times \begin{array}{c} \text{Block Pointer} \\ \text{size} \end{array} \right\} \leq \text{Disk Block size}$$



Topic : Order of leaf node of B+ Tree

- Let 'm' is the order of a leaf node of B+ Tree
 - Maximum number of keys a leaf node can have is = m
 - Minimum number of keys a non-root leaf node must have is = $\lceil m/2 \rceil$
 - If leaf node is also a root node then can have minimum '1' key
 - There will be exactly one block pointer in each leaf node, it will point to the next leaf node of B+ tree. (because of this pointer range query is more efficient in B+ tree)

Search for the records
with search key values
from 100 to 200



Topic : NOTE



- If only one order is given in the question, then we will use the same order for both leaf node as well as for internal node, But if only one order is given then things will be different

Eg:

Consider a B+ tree of order = n

{ Only one order is given }

Korth

- * Maximum number of child pointer an internal node can have = n
- * Maximum number of keys an internal node can have = $(n-1)$
- * Minimum number of child pointer an internal node must have = $\lceil \frac{n}{2} \rceil$
(for non-root)
- * Minimum number of keys an internal node must have = $\lceil \frac{n}{2} \rceil - 1$
(for non-root)

Navathe

- * Maximum number of child pointer an internal node can have = n
- * Maximum number of keys an internal node can have = $(n-1)$
- * Minimum number of child pointer an internal node must have = $\lceil \frac{n}{2} \rceil$
(for non-root)
- * Minimum number of keys an internal node must have = $\lceil \frac{n}{2} \rceil - 1$
(for non-root)

Eg.:

Consider a B+ tree of order = n

{ only one order is given }

Korth

Navathe

* Maximum number of keys
a leaf node can have = $(n-1)$

* Maximum number of keys
a leaf node can have = $(n-1)$

* Minimum number of keys
a leaf node must have
(for non-root) = $\lceil \frac{(n-1)}{2} \rceil$

* Minimum number of keys
a leaf node must have
(for non-root) = $\lceil \frac{n}{2} \rceil$

HoNo
#Q.

B tree



- a) Find the minimum number of nodes required in B tree of order = p and height = h
- b) Find the minimum number of child pointers required in B tree of order = p and height = h
- c) Find the minimum number of keys required in B tree of order = p and height = h
- d) Find the maximum number of nodes possible in B tree of order = p and height = h
- e) Find the maximum number of child pointers possible in B tree of order = p and height = h
- f) Find the maximum number of keys possible in B tree of order = p and height = h

★ No. of nodes, no. of keys and no. of record pointers
Will be maximum, when node occupancy is maximum

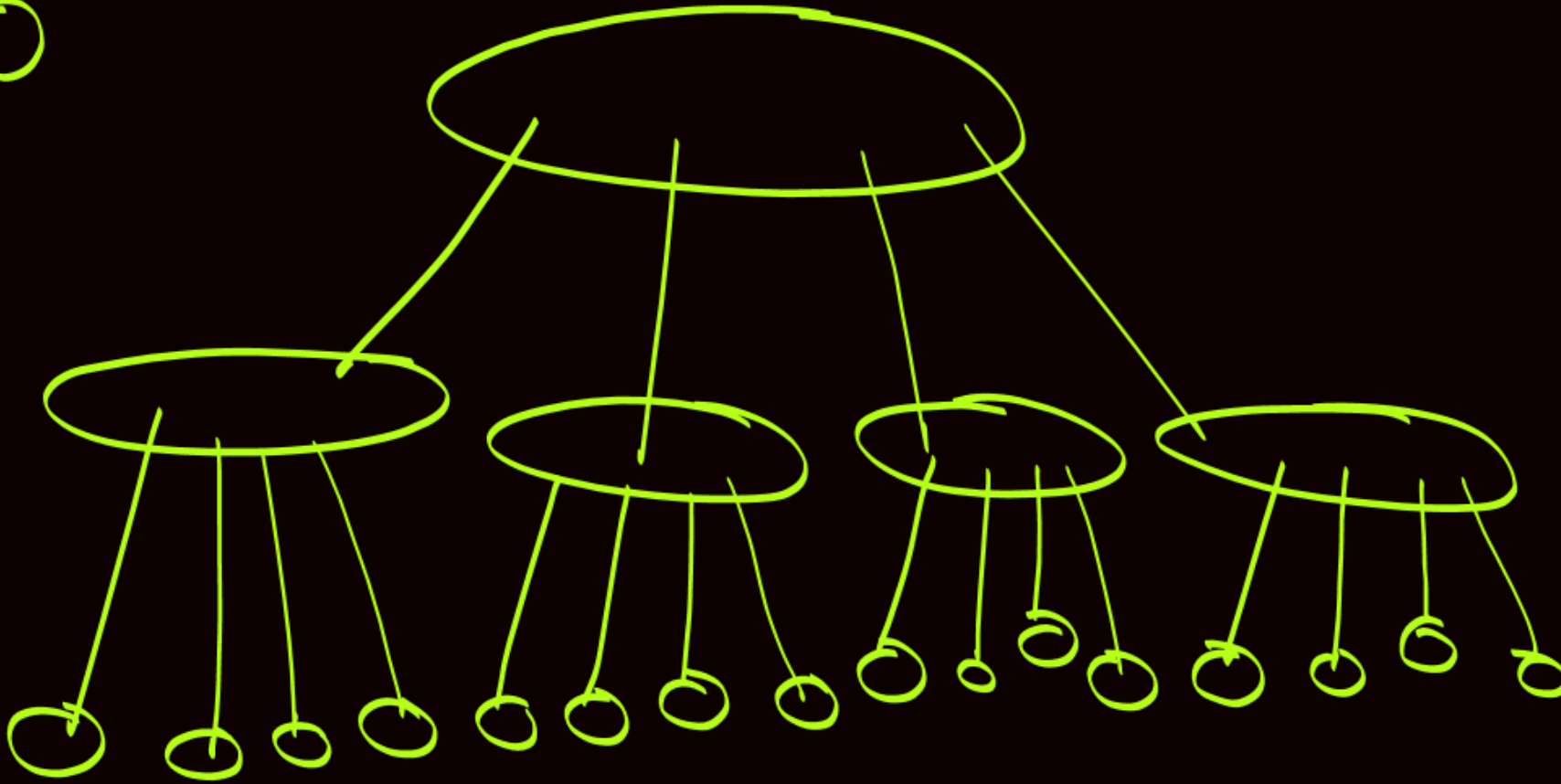
★ No. of nodes, no. of keys and no. of record pointers
Will be minimum when node occupancy is minimum

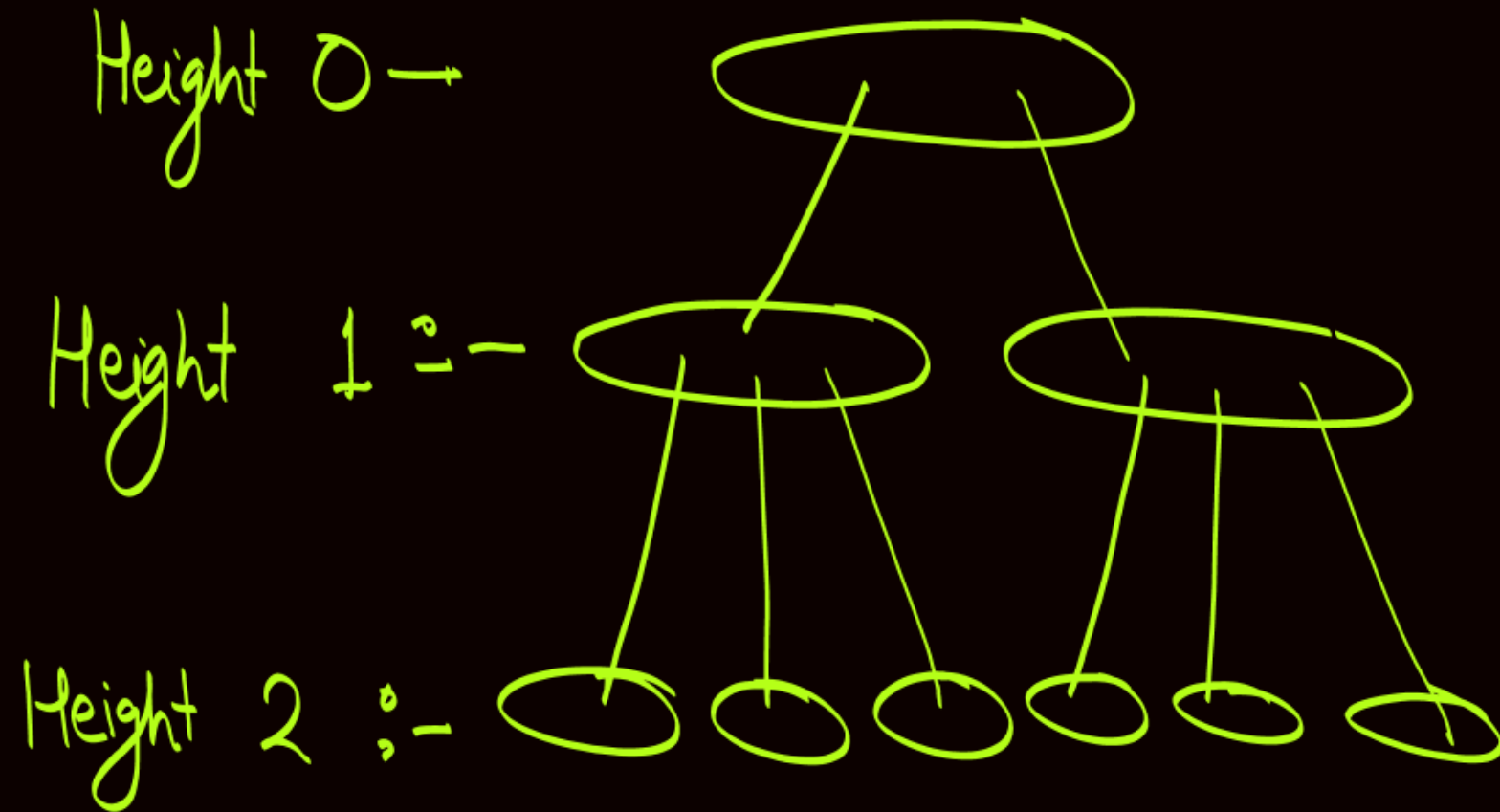
Order = 4

Height = 0

Height = 1

✓ Height = 2





Order = 6

Min no. of nodes
in B tree
of height = 2

Min no. of
Child pointers
a root node can have = 2

Min no. of child
pointers a non
root node must have = $\lceil \frac{6}{2} \rceil = 3$

W.r.t. B tree of order = P

Height/Level	Minimum Number of nodes	Minimum Number of Child Pointer	Minimum number of Keys	Maximum Number of nodes	Maximum Number of Child Pointer	Maximum Number of Keys
At 0/1 (root)	1	2	1	1	P	(P-1)
At 1/2 (Children of root)	2	$2 \times \left\lceil \frac{P}{2} \right\rceil$	$2 \times \left(\left\lceil \frac{P}{2} \right\rceil - 1 \right)$ $2 \times \left\lceil \frac{P}{2} \right\rceil \times \left(\left\lceil \frac{P}{2} \right\rceil - 1 \right)$	P	$P \times P = P^2$	$P \times (P-1)$
At 2/3 (Children of non-root node)	$2 \times \left\lceil \frac{P}{2} \right\rceil = 2 \times \left\lceil \frac{P}{2} \right\rceil^1$	$2 \times \left\lceil \frac{P}{2} \right\rceil \times \left(\left\lceil \frac{P}{2} \right\rceil \right) = 2 \left\lceil \frac{P}{2} \right\rceil^2$	$2 \times \left\lceil \frac{P}{2} \right\rceil \times \left(\left\lceil \frac{P}{2} \right\rceil - 1 \right)$ $2 \times \left\lceil \frac{P}{2} \right\rceil^1 \times \left(\left\lceil \frac{P}{2} \right\rceil - 1 \right)$	P^2	$P^2 \times P = P^3$	$P^2 \times (P-1)$
At 3/4 (Children of non-root node)	$2 \times \left\lceil \frac{P}{2} \right\rceil^2$	$2 \times \left\lceil \frac{P}{2} \right\rceil^2 \times \left\lceil \frac{P}{2} \right\rceil = 2 \times \left\lceil \frac{P}{2} \right\rceil^3$	$2 \times \left\lceil \frac{P}{2} \right\rceil^2 \times \left(\left\lceil \frac{P}{2} \right\rceil - 1 \right)$	P^3	$P^3 \times P = P^4$	$P^3 \times (P-1)$
⋮	⋮	⋮	⋮	⋮	⋮	⋮
h/(h+1)	$2 \times \left\lceil \frac{P}{2} \right\rceil^{h-1}$	$2 \times \left\lceil \frac{P}{2} \right\rceil^h$	$2 \times \left\lceil \frac{P}{2} \right\rceil^{h-1} \times \left(\left\lceil \frac{P}{2} \right\rceil - 1 \right)$	P^h	$P^h \times P = P^{h+1}$	$P^h \times (P-1)$

	Height/Level	Minimum Number of nodes	Minimum Number of Child Pointers	Minimum number of Keys	Maximum Number of nodes	Maximum Number of Child Pointers	Maximum Number of Keys
	At 0/1 (root)	1	2	1	1	P	(P-1)
	At 1/2 (children of root)	2	$2 \times \lceil \frac{P}{2} \rceil$	$2 \times (\lceil \frac{P}{2} \rceil - 1)$	P	$P \times P = P^2$	$P \times (P-1)$
	At 2/3 (children of non-root node)	$2 \times \lceil \frac{P}{2} \rceil = 2 \times \lceil \frac{P}{2} \rceil^2$	$2 \times \lceil \frac{P}{2} \rceil \times (\lceil \frac{P}{2} \rceil) = 2 \lceil \frac{P}{2} \rceil^2$	$2 \times \lceil \frac{P}{2} \rceil \times (\lceil \frac{P}{2} \rceil - 1)$	P^2	$P^2 \times P = P^3$	$P^2 \times (P-1)$
	At 3/4 (children of non-root node)	$2 \times \lceil \frac{P}{2} \rceil^2$	$2 \times \lceil \frac{P}{2} \rceil^2 \times \lceil \frac{P}{2} \rceil = 2 \times \lceil \frac{P}{2} \rceil^3$	$2 \times \lceil \frac{P}{2} \rceil^2 \times (\lceil \frac{P}{2} \rceil - 1)$	P^3	$P^3 \times P = P^4$	$P^3 \times (P-1)$
	...	...	...	...	...	...	...
	At h/(h+1)	$2 \times \lceil \frac{P}{2} \rceil^{h-1}$	$2 \times \lceil \frac{P}{2} \rceil^h$	$2 \times \lceil \frac{P}{2} \rceil^{h-1} \times (\lceil \frac{P}{2} \rceil - 1)$	P^h	$P^h \times P = P^{h+1}$	$P^h \times (P-1)$

Writ. B tree of
Order = P

Summation of all the terms will give us the minimum no. of nodes a B tree of height 'h' & order 'P' must have

Same for others



2 mins Summary



✓ **Topic**

Dynamic Multi level index

✓ **Topic**

Structure of B tree

✓ **Topic**

Structure of B+ tree

THANK - YOU